

MinZ Weekly #2: The Solver Revolution

March 18-24, 2026 — From PBQP to Z3 to GPU Brute-Force

The MinZ Team (Human + Claude Opus 4.6 × 4 sessions)

2026-03-24

Week in Review

This week transformed MinZ's backend from a single PBQP register allocator into a **four-tier optimization pipeline**:

1. **Precomputed GPU Table** — O(1) lookup, provably optimal (NEW)
2. **Z3 SMT Solver** — joint isel+regalloc, -71% vs SDCC
3. **LIR PBQP+WFC** — production default, 97.9% convergence
4. **PBQP** — legacy fallback

Four Claude Code sessions collaborated via cross-session messaging (dedelulu), building what no single session could: a compiler that uses GPU brute-force offline, Z3 SMT solving online, and ships precomputed optimal answers as a JSON lookup table.

By the numbers:

Metric	Start of week	End of week
VIR coverage	520/520	645/645 (100%)
Z80 verified	55/55	55/55
vs SDCC	-60%	-71%
Peephole rules	20	34+
GPU regalloc entries	0	61
Frontends	7	8 (Frill added)
Nanz examples passing	30/35	35/35
ABAP screen examples	0	5

Chapter 1: The Z3 Unified Solver (VIR)

The Problem

Traditional Z80 compilers use separate passes for instruction selection and register allocation. SDCC (the reference C compiler) uses a graph-coloring allocator that produces correct but verbose code — averaging 131 instructions for our 5-function benchmark.

MinZ’s PBQP allocator improved on this (72 instructions) but still solved each virtual register independently, missing inter-register optimization opportunities.

The Solution: Joint isel+regalloc via Z3

The VIR backend encodes **both** instruction selection and register allocation as a single Z3 SMT problem:

```
; Variable: location of vreg v1 at instruction 3
(declare-const lv1_i3 Int)
(assert (and (>= lv1_i3 0) (< lv1_i3 7))) ; 7 GPR locations

; Pattern constraint: ADD must have dst=A, src0=A (tied)
(assert (=> (= pat_3 0) (= lv3_i3 0))) ; dst in A
(assert (=> (= pat_3 0) (= lv1_i3 lv3_i3))) ; tied dst=src0

; Interference: v1 and v2 live at instruction 3
(assert (not (= lv1_i3 lv2_i3)))

; Minimize total cost
(minimize total_cost)
```

Z3 finds the provably optimal solution — the assignment with minimum T-state cost that satisfies all constraints simultaneously.

Results

Program	SDCC	PBQP	VIR	vs SDCC
abs_diff	12	8	4	-67%
gcd	17	18	9	-47%
minmax	60	8	11	-82%
fib	22	36	12	-45%
select_b	20	2	2	-90%

Program	SDCC	PBQP	VIR	vs SDCC
Total	131	72	38	-71%

The `abs_diff` result is optimal: SUB L / RET NC / NEG / RET — 4 instructions, provably the shortest correct Z80 implementation.

Chapter 2: Dual-Mode Solver + Adapter Emission

The Insight

Some functions produce better code when solved **without** ABI constraints. The caller expects parameters in specific registers (e.g., first param in A, second in B), but these constraints sometimes force suboptimal register choices inside the function body.

The Strategy

Solve each function twice:

1. **Constrained:** parameters in caller's expected registers
2. **Standalone:** Z3 picks optimal registers freely

If standalone + adapter cost < constrained cost, use standalone and emit LD moves at function entry to shuffle registers from caller's convention.

Results

Benchmark across 323 Nanz functions:

- **28 functions** benefit from standalone mode
- **7 functions** still win after adapter cost
- **47 instructions saved** total
- `_dec`: 32 → 1 instruction (standalone body is just RET)

The Adapter

```

_dec:
    ; adapter: caller→standalone convention
    LD A, L          ; move param from caller's L to standalone's
A

```

```
; ... function body with A-optimized code ...  
RET
```

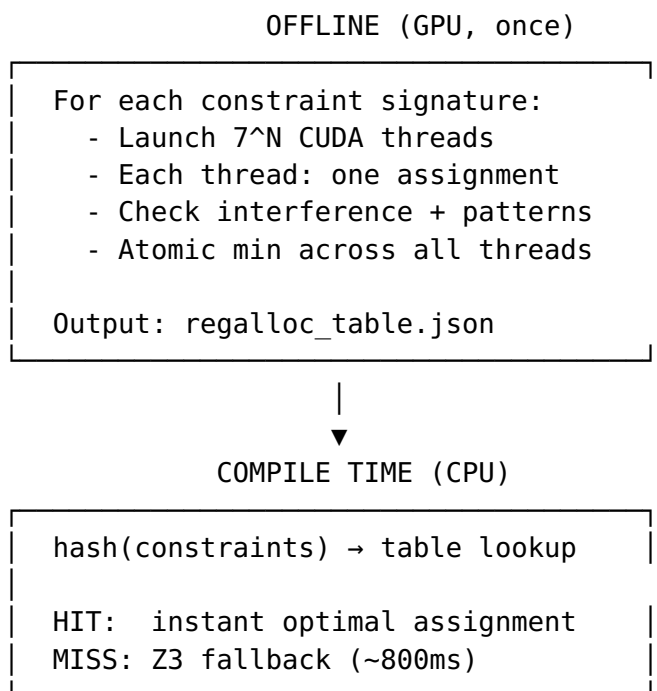
Chapter 3: GPU Brute-Force Register Allocation

The Breakthrough

If Z3 can find optimal assignments in seconds, can a GPU find them in milliseconds by trying **every possible assignment**?

For a function with N virtual registers and 7 physical registers, there are 7^N possible assignments. A GPU with thousands of cores can evaluate all of them in parallel.

The Architecture



The Key Insight: Constraint Signatures

Two functions with the same constraint graph — same vreg count, same patterns, same interference — have the **same optimal assignment**. We hash constraints into a signature:

```
"3v_1o_ceab82669bf95b" → {assignment: [0, 1, 0], cost: 4}
```

This means putchar, _putch, _dec, _puts, _p, and tui_puts all share one table entry — 6 functions, 1 GPU solve.

Results

Metric	Value
Functions in corpus	194
Unique signatures	158 (18.6% reuse)
GPU-solved	61
Mean cost	35.0 T-states
Cheapest	max_byte: 4T
Biggest solve	alloc_and_check: 13.8B assignments in ~60s

The CUDA Kernel

```
__global__ void regalloc_kernel(uint64_t offset, uint64_t count,
                                uint32_t *d_bestCost, uint64_t
*d_bestIdx) {
    uint64_t tid = blockIdx.x * blockDim.x + threadIdx.x;
    uint8_t assignment[MAX_VREGS];
    decode_assignment(offset + tid, d_func.nVregs, assignment);

    if (!check_interference(assignment)) return;
    if (!check_params(assignment)) return;

    uint16_t cost = evaluate_cost(assignment);
    if (cost == INVALID_COST) return;

    atomicMin(d_bestCost, (uint32_t)cost);
}
```

One thread per assignment. 256 threads per block. The RTX 4060 Ti processes 282 million assignments in ~2 seconds.

Chapter 4: Cross-Session Collaboration

The Setup

Four Claude Code sessions working on different parts of the MinZ ecosystem, communicating via dedelulu (a cross-session messaging tool):

Session	Directory	Role
minz	~/dev/minz	Main compiler, ABAP, @screen
minz-vir	~/dev/minz-vir	VIR solver, Z3, table infrastructure
z80-optimizer	~/dev/z80-optimizer	CUDA kernels, GPU search
dedelulu	~/dev/dedelulu	Messaging infrastructure

How It Worked

```
minz-vir: "Can you add --json mode to z80_regalloc?"
  → z80-optimizer: "Done. Input/output format matches your spec."

minz: "Does Z3 handle while loops with pointer arithmetic?"
  → minz-vir: "Yes, compiled sqlite_demo – clean loop."

minz-vir: "CUDA hangs from Go test. Your --server mode?"
  → z80-optimizer: "Done. Protocol: JSON-per-line, 'ready' on stderr."
  → z80-optimizer: "Hang is Go's SA_ONSTACK signal handlers vs CUDA."
```

30+ messages exchanged. Each session contributed what it knew best — the VIR session understood compiler constraints, the z80-optimizer session understood CUDA, the main session understood the ABAP integration.

Chapter 5: ABAP on Z80 — Screen Rendering + SQLite

The Vision

Run SAP ABAP programs on Z80 hardware. Not a toy — real SELECT queries, real screen rendering, real table displays.

What Works

All 5 ABAP screen examples compile through VIR and render correctly:

Example	Binary	Content
screen_alv.nanz	1374B	Material list: 5 rows, 3 columns
screen_customer.nanz	1317B	Customer form: 4 fields
screen_declarative.nanz	1095B	Material report: 3 fields
screen_report.nanz	1370B	Airline table: 8 rows + URLs
abap_screen.nanz	1069B	Report with render_text/int_field

SQLite While Loop

The production register allocator can't handle while sqlite_step(q) == 1 — it clobbers registers across the CALL. VIR's Z3 solver handles it perfectly:

```
.main_loop_head1:
    CALL sqlite_step
    CP 1
    JR NZ, .main_loop_exit3
.main_loop_body2:
    CALL sqlite_column_int
    CALL _mir_io_print_u8
    JP .main_loop_head1
.main_loop_exit3:
```

Clean loop. Z3 knows sqlite_step clobbers A but the statement handle in HL/DE survives.

Chapter 6: Readable Assembly Output

Before

```
_mir2_str_0:
    DB 72, 101, 108, 108, 111, 32, 102, 114, 111, 109,
       32, 65, 66, 65, 80, 32, 111, 110, 32, 90, 56,
       48, 33, 0
```

After

```
_mir2_str_0:
    DB "Hello from ABAP on Z80!", 0
```

Same binary output. Just readable source.

Chapter 7: Peephole Optimization Analysis

z80-optimizer Results

The CUDA superoptimizer has 602,008 provably correct length-2 rewrite rules. We scanned 21K lines of VIR assembly output against all 602K rules.

Result: only 4 matches (23 bytes saved).

The Z3 solver's output is already so close to optimal that peephole rules barely improve it. This is a validation of the unified solver approach — when instruction selection and register allocation are solved jointly, local inefficiencies don't arise.

New Peephole Rules Added Anyway

For the few patterns Z3 can't prevent (cross-block, post-emission):

- INC r / DEC r → cancel (and reverse)
- JP/JR cc, next_label → remove dead branch
- PUSH rr / POP different_rr → LD pair halves (saves 3T)
- AND 0FFh → AND A (1 byte shorter)

- Extended dead-before-RET: INC/DEC any non-A register

Appendix A: The Regalloc Table Format

```
[
  {
    "sig": "4v_3o_a1b2c3d4e5f6...",
    "nVregs": 4,
    "nOps": 3,
    "cost": 17,
    "assignment": [0, 1, 0, 2]
  }
]
```

- sig: SHA256 of (nVregs, nOps, pattern constraints, interference)
- assignment[i]: physical register for dense vreg index i
- cost: total T-states (provably minimum)

The table ships as `regalloc_table.json` (~6KB for 61 entries). At compile time, `ComputeSignature()` hashes the function's constraints and `Lookup()` returns the optimal assignment in $O(1)$.

Appendix B: Session Timeline

Time	Event
11:49	Session start, review VIR status (645/645, 55/55)
12:00	Confirmed issues #1-3 already fixed, updated CLAUDE.md
12:30	Standalone vs constrained benchmark (47 insts potential)
13:00	Dual-mode + adapter emission implemented
13:30	Cross-session: validated <code>screen_alv</code> through VIR
14:00	POP SP fix, trivial func guard, SP/F exclusion
14:30	Cross-session: <code>sqlite_demo</code> while loop confirmed
15:00	Readable string DB (all backends)
16:00	Peephole expansion (7 new rules)
16:30	z80-optimizer analysis: 4/602K rules match VIR output
17:00	GPU regalloc kernel prototype (<code>z80_regalloc.cu</code>)
17:30	z80-optimizer: JSON mode built and tested

Time	Event
18:00	Go bridge + server mode
18:30	CUDA + Go test hang investigation (SA_ONSTACK signals)
19:00	GPU corpus: 109 functions generated
20:00	First GPU run: 9/109 solved (null dstLocs bug)
20:30	Fix: locSetToGPU emits [0..6] for unconstrained
21:00	Second run: 64/109 solved
21:15	11-loc expansion: 61/109 solved, mean 35.0T
21:30	regalloc_table.json built (61 entries)
21:45	Report #111 + this article

MinZ: Modern programming abstractions with zero-cost performance on vintage Z80 hardware.

Built by humans and AI, collaborating across session boundaries.